

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

-

CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

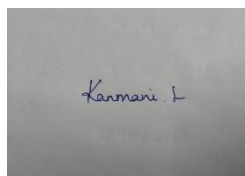
Industry Problem-Solving Task

Code of Conduct:

I Kanmani L/192511414 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

1. SSL/TLS Overhead and Web Server Performance Optimization

A web server uses TLS 1.3 for secure communication. Each HTTP response carries 2000 bytes of payload. TLS adds 25 bytes (record header + authentication tag) and handshake overhead of 3 KB per session. The server handles 10,000 requests per second, with 1,000 new TLS handshakes per second.

Task:

- A. Calculate the total data transmitted per second including TLS overhead.
 - B. Determine the percentage overhead due to TLS encryption.
 - C. Estimate bandwidth consumed due to handshake overhead per second.
 - D. Analyze the trade-off between security and latency, and propose optimization techniques (e.g., session resumption, HTTP/2, or QUIC).
-

2. PGP-Based Email Encryption Load in Enterprise System

An enterprise email server encrypts emails using PGP. Each email contains a 10 MB attachment. PGP introduces 800 bytes overhead for key encryption and signatures. The system processes 5,000 emails per day.

Task:

- A. Calculate the total data transmitted per day including overhead.
 - B. Determine the percentage increase in storage due to PGP.
 - C. Analyze the computational overhead during encryption/decryption.
 - D. Recommend optimization strategies (e.g., compression, batching, or hardware acceleration).
-

3. IPSec Transport vs Tunnel Mode Performance Comparison

A company compares IPSec transport mode and tunnel mode for securing internal communications. Original packet size = 1000 bytes payload + 20-byte header. Transport mode adds 30 bytes, while tunnel mode adds 50 bytes overhead. Traffic volume = 20,000 packets per second.

Task:

- A. Compute total packet size for both modes.
 - B. Calculate bandwidth usage for each mode per second.
 - C. Compare efficiency and security benefits of both modes.
 - D. Recommend the appropriate mode for internal vs external communication with justification.
-

4. S/MIME Certificate Management in Large Organization

An organization manages 12,000 employees using S/MIME certificates. Each certificate is 2 KB, and certificate renewal happens every year. During renewal, both old and new certificates are stored simultaneously for 1 month.

Task:

- A. Calculate total storage required during the renewal period.
 - B. Estimate certificate distribution overhead across the network.
 - C. Analyze challenges in certificate lifecycle management at scale.
 - D. Propose optimization strategies (e.g., automation, centralized PKI, or short-lived certificates).
-

5. Email Encryption Latency in Cloud-Based Systems

A cloud email service encrypts outgoing emails using AES (data) and RSA (key exchange). AES encryption takes 0.2 ms per MB, and RSA key exchange takes 5 ms per email. Each email has 8 MB attachment. The system processes 8,000 emails per hour.

Task:

- A. Calculate total encryption time per email.
- B. Estimate total processing time per hour.
- C. Analyze the performance bottleneck between symmetric and asymmetric operations.
- D. Recommend optimization strategies (e.g., ECC adoption, parallel processing, or hardware acceleration).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. SSL/TLS Overhead and Web Server Performance Optimization

A web server uses TLS 1.3 for secure communication. Each HTTP response carries 2000 bytes of payload. TLS adds 25 bytes (record header + authentication tag) and handshake overhead of 3 KB per session. The server handles 10,000 requests per second, with 1,000 new TLS handshakes per second.

Task:

- Calculate the total data transmitted per second including TLS overhead.
- Determine the percentage overhead due to TLS encryption.
- Estimate bandwidth consumed due to handshake overhead per second.
- Analyze the trade-off between security and latency, and propose optimization techniques (e.g., session resumption, HTTP/2, or QUIC).

Answer:

Given

- HTTP payload per response = 2000 bytes
- TLS overhead per response = 25 bytes
- Requests per second = 10,000
- New TLS handshakes per second = 1,000
- Handshake overhead = 3 KB/session

A. Total data transmitted per second including TLS overhead

Data per response:

$$2000 + 25 = 2025 \text{ bytes}$$

For 10,000 requests/sec:

$$2025 \times 10,000 = 20,250,000 \text{ bytes/sec}$$

So, application data + per-record TLS overhead:

$$20.25 \text{ MB/s}$$

In bits:

$$20,250,000 \times 8 = 162,000,000 \text{ bits/sec } 162 \text{ Mbps}$$

Including the handshake overhead from part C:

$$20.25 + 3 = 23.25 \text{ MB/s}$$

or

$$23.25 \times 8 = 186 \text{ Mbps}$$

B. Percentage overhead due to TLS encryption

TLS overhead per response = 25 bytes.

$$\text{Overhead \%} = \frac{25}{2000} \times 100 = 1.25\%$$

Thus, TLS adds 1.25% overhead to the HTTP payload.

C. Bandwidth consumed due to handshake overhead

Each handshake requires 3 KB.

For 1,000 new handshakes/sec:

$$3 \text{ KB} \times 1000 = 3000 \text{ KB/s} = 3 \text{ MB/s}$$

In bits:

$$3 \times 8 = 24 \text{ Mbps}$$

Therefore, TLS handshakes consume approximately 3 MB/s (24 Mbps) of bandwidth.

D. Security vs. latency and optimization

TLS provides important confidentiality, authentication, and integrity, but handshakes introduce additional processing, network round trips, and bandwidth overhead. A high rate of new handshakes can therefore increase latency and server CPU usage.

Optimization techniques:

1. Session Resumption: Reuse an existing TLS session instead of performing a complete handshake for every connection. This reduces handshake latency and CPU overhead.
2. HTTP/2: Multiplexes multiple requests over a single TLS connection, reducing the need for repeated connections and handshakes.
3. QUIC/HTTP/3: Uses QUIC with TLS 1.3 and reduces connection-establishment latency while supporting efficient multiplexing.
4. Persistent Connections: Keep TLS connections open and serve multiple requests through them.
5. Connection Pooling: Reuse established secure connections between the client and server.

Final Summary

Parameter	Result
Payload traffic	20 MB/s
TLS record overhead	0.25 MB/s
Data including TLS record overhead	20.25 MB/s
TLS encryption overhead	1.25%
Handshake bandwidth	3 MB/s (24 Mbps)
Total including handshake overhead	23.25 MB/s (186 Mbps)

TLS introduces relatively small per-response overhead (1.25%), but frequent handshakes can significantly increase bandwidth, latency, and CPU consumption. Session resumption, HTTP/2, persistent connections, and QUIC/HTTP/3 can maintain strong security while improving web-server performance.

2. PGP-Based Email Encryption Load in Enterprise System

An enterprise email server encrypts emails using PGP. Each email contains a 10 MB attachment. PGP introduces 800 bytes overhead for key encryption and signatures. The system processes 5,000 emails per day.

Task:

- Calculate the total data transmitted per day including overhead.
- Determine the percentage increase in storage due to PGP.
- Analyze the computational overhead during encryption/decryption.
- Recommend optimization strategies (e.g., compression, batching, or hardware acceleration).

Answer:

Given

- Attachment size = 10 MB
- PGP overhead per email = 800 bytes
- Emails processed per day = 5,000

Assume:

1 MB=1,000,000 bytes

So:

10 MB=10,000,000 bytes

A. Total data transmitted per day including PGP overhead

Data for one email:

$$10,000,000 + 800 = 10,000,800 \text{ bytes}$$

For 5,000 emails:

$$10,000,800 \times 5000 = 50,004,000,000 \text{ bytes}$$

Therefore:

$$50,004,000,000 \text{ bytes/day}$$

or approximately:

$$50.004 \text{ GB/day}$$

The original data is:

$$10,000,000 \times 5000 = 50,000,000,000$$

$$= 50 \text{ GB/day.}$$

So PGP adds 4,000,000 bytes = 4 MB/day.

B. Percentage increase in storage due to PGP

$$\text{Percentage Increase} = \frac{\text{Original data PGP overhead}}{\text{Original data}} \times 100 = \frac{4,000,000}{50,000,000,000} \times 100 = 0.008\%$$

Thus, PGP increases storage by only 0.008%, which is very small compared with the 10 MB attachment size.

C. Computational overhead during encryption/decryption

PGP uses a hybrid encryption approach:

- Symmetric encryption encrypts the 10 MB attachment and is computationally efficient.
- Public-key encryption is mainly used to encrypt the small session key.
- Digital signatures require additional computation for hashing and signing.

Therefore, for 5,000 emails per day, the main computational workload comes from symmetric encryption/decryption of the 10 MB attachments, while public-key operations and signatures add comparatively smaller overhead.

The exact CPU time cannot be calculated from the given data because it depends on the algorithm, processor speed, implementation, and key size.

D. Optimization Strategies

1. Compression: Compress attachments before encryption to reduce the amount of data that needs to be encrypted and transmitted.
2. Batching: Process multiple encryption/decryption operations together to reduce repeated processing overhead.
3. Hardware Acceleration: Use cryptographic hardware or CPU acceleration such as AES-NI to speed up encryption.
4. Efficient Algorithms: Use efficient symmetric algorithms such as AES for large attachments.
5. Session/Key Reuse: Where appropriate, reduce repeated expensive public-key operations by using efficient key-management techniques.

Final Answer

Parameter	Result
Original data/day	50 GB
PGP overhead/email	800 bytes
PGP overhead/day	4 MB
Total data/day	50.004 GB
Storage increase	0.008%

PGP provides strong confidentiality and authentication with very little storage overhead (0.008%). The major performance cost is the computational work involved in encrypting and decrypting large attachments, which can be reduced through compression, batching, efficient symmetric encryption, and hardware acceleration.

3. IPSec Transport vs Tunnel Mode Performance Comparison

A company compares IPSec transport mode and tunnel mode for securing internal communications. Original packet size = 1000 bytes payload + 20-byte header.

Transport mode adds 30 bytes, while tunnel mode adds 50 bytes overhead. Traffic volume = 20,000 packets per second.

Task:

- A. Compute total packet size for both modes.
- B. Calculate bandwidth usage for each mode per second.
- C. Compare efficiency and security benefits of both modes.
- D. Recommend the appropriate mode for internal vs external communication with justification.

Answer:

Given

- Payload = 1000 bytes
- Original IP header = 20 bytes
- Original packet size:

$$1000+20=1020 \text{ bytes}$$

- Transport mode overhead = 30 bytes
- Tunnel mode overhead = 50 bytes
- Traffic = 20,000 packets/second

A. Total packet size

Transport Mode:

$$1020+30=1050 \text{ bytes}$$

Tunnel Mode:

$$1020+50=1070 \text{ bytes}$$

B. Bandwidth usage per second

Transport Mode

$$1050 \times 20,000 = 21,000,000 \text{ bytes/sec}$$

In bits:

$$21,000,000 \times 8 = 168 \text{ Mbps}$$

Tunnel Mode

$$1070 \times 20,000 = 21,400,000 \text{ bytes/sec}$$

In bits:

$$21,400,000 \times 8 = 171.2 \text{ Mbps}$$

Therefore:

- Transport Mode = 21 MB/s = 168 Mbps
- Tunnel Mode = 21.4 MB/s = 171.2 Mbps

C. Efficiency and security comparison

Feature	Transport Mode	Tunnel Mode
Packet size	1050 bytes	1070 bytes
Bandwidth	168 Mbps	171.2 Mbps
Efficiency	Higher	Slightly lower
Original IP header	Remains visible	Protected
Security	Suitable for host-to-host communication	Provides greater privacy
Common use	Internal communication	VPN/external communication

Transport mode is more efficient because it adds only 30 bytes of overhead compared with 50 bytes for tunnel mode.

Tunnel mode provides stronger protection because the entire original IP packet is encapsulated and protected, hiding the original source and destination addresses from external observers.

D. Recommendation

For internal communication: Use IPSec Transport Mode because internal host-to-host communication generally benefits from lower overhead and better bandwidth efficiency.

For external communication: Use IPSec Tunnel Mode, especially for VPNs and site-to-site communication, because it protects the entire original IP packet and hides the internal addressing information.

Transport mode uses 168 Mbps, while tunnel mode uses 171.2 Mbps. Thus, transport mode is slightly more bandwidth-efficient, while tunnel mode provides greater privacy and is better suited for securing communication across untrusted external networks.

4. S/MIME Certificate Management in Large Organization

An organization manages 12,000 employees using S/MIME certificates. Each certificate is 2 KB, and certificate renewal happens every year. During renewal, both old and new certificates are stored simultaneously for 1 month.

Task:

- Calculate total storage required during the renewal period.
- Estimate certificate distribution overhead across the network.
- Analyze challenges in certificate lifecycle management at scale.
- Propose optimization strategies (e.g., automation, centralized PKI, or short-lived certificates).

Answer:

Given

- Number of employees = 12,000

- Certificate size = 2 KB
- Renewal frequency = 1 year
- Old and new certificates stored simultaneously = 1 month

Assume:

1 KB=1000 bytes

So each certificate = 2,000 bytes.

A. Total storage required during the renewal period

Normally, one certificate is stored for each employee:

$12,000 \times 2 \text{ KB} = 24,000 \text{ KB} = 24 \text{ MB}$

During the 1-month renewal period, both old and new certificates are stored:

$24 \text{ MB} \times 2 = 48 \text{ MB}$

Therefore, the total storage required during renewal is 48 MB.

B. Certificate distribution overhead across the network

The new certificate must be distributed to all 12,000 employees:

$12,000 \times 2 \text{ KB} = 24,000 \text{ KB} = 24 \text{ MB}$

In bits:

$24 \times 8 = 192 \text{ Mb}$

Therefore, distributing all renewed certificates requires approximately 24 MB (192 Mb) of network data.

If the distribution is completed over one day:

$86,400 / 192 \text{ Mb} \approx 2.22 \text{ Kbps}$

The bandwidth itself is small, but simultaneous certificate requests from 12,000 employees can create a significant PKI server and authentication load.

C. Challenges in certificate lifecycle management

Managing certificates for 12,000 employees creates several challenges:

1. Large-scale renewal: Thousands of certificates must be renewed without interruption.
2. Expiration management: Expired certificates can prevent users from sending or receiving secure emails.
3. Revocation: Certificates of employees who leave the organization must be revoked quickly.
4. Key management: Private keys must be securely generated, stored, and protected.
5. Distribution: New certificates must reach all users and devices reliably.
6. Tracking: The organization must maintain accurate records of certificate status and expiration dates.

D. Optimization Strategies

1. Automation: Automatically generate, renew, distribute, and revoke certificates to reduce manual work.
2. Centralized PKI: Use a centralized Public Key Infrastructure to manage certificates, keys, and policies.
3. Short-lived Certificates: Use certificates with shorter validity periods and automated renewal to reduce the impact of compromised certificates.
4. Staggered Renewal: Renew certificates in batches instead of renewing all 12,000 simultaneously.
5. Centralized Monitoring: Monitor certificate expiration and automatically alert administrators before certificates expire.

Final Summary

Parameter	Result
Employees	12,000
Certificate size	2 KB
Normal storage	24 MB
Storage during renewal	48 MB
New certificate distribution	24 MB (192 Mb)

S/MIME certificate management at this scale requires automation and centralized PKI to handle renewal, distribution, revocation, and monitoring efficiently. Although the certificate data itself requires relatively little storage and bandwidth, automated lifecycle management is essential to avoid service disruptions and administrative overhead.

5. Email Encryption Latency in Cloud-Based Systems

A cloud email service encrypts outgoing emails using AES (data) and RSA (key exchange). AES encryption takes 0.2 ms per MB, and RSA key exchange takes 5 ms per email. Each email has 8 MB attachment. The system processes 8,000 emails per hour.

Task:

- A. Calculate total encryption time per email.
- B. Estimate total processing time per hour.
- C. Analyze the performance bottleneck between symmetric and asymmetric operations.
- D. Recommend optimization strategies (e.g., ECC adoption, parallel processing, or hardware acceleration).

Answer:

Given

- AES encryption time = 0.2 ms/MB
- Attachment size = 8 MB
- RSA key exchange time = 5 ms/email
- Emails processed = 8,000 emails/hour

A. Total encryption time per email

AES encryption time:

$$0.2 \times 8 = 1.6 \text{ ms}$$

RSA key exchange time:

$$5 \text{ ms}$$

Therefore:

$$1.6 + 5 = 6.6 \text{ ms/email}$$

B. Total processing time per hour

Number of emails = 8,000.

$$6.6 \times 8000 = 52,800 \text{ ms}$$

Convert to seconds:

$$100052,800 = 52.8 \text{ seconds}$$

Therefore, the total sequential encryption processing time is 52.8 seconds per hour.

C. Performance bottleneck

AES takes:

1.6 ms

RSA takes:

5 ms

Percentage contribution of RSA:

$6.65 \times 100 \approx 75.76\%$

Percentage contribution of AES:

$6.61.6 \times 100 \approx 24.24\%$

Therefore, RSA key exchange is the main performance bottleneck, contributing approximately 75.76% of the encryption time per email. AES is faster and is mainly responsible for encrypting the large 8 MB attachment.

D. Optimization Strategies

1. **ECC Adoption:** Replace RSA with Elliptic Curve Cryptography (ECC) for key exchange. ECC provides comparable security with smaller keys and generally lower computational overhead.
2. **Parallel Processing:** Process multiple emails simultaneously using multiple CPU cores or cloud instances.
3. **Hardware Acceleration:** Use AES hardware acceleration such as AES-NI to speed up symmetric encryption.
4. **Efficient Key Management:** Reuse secure session keys where appropriate to reduce repeated asymmetric operations.
5. **Cloud Scaling:** Use additional computing instances during periods of high email traffic.

Final Summary

Parameter	Result
AES time/email	1.6 ms
RSA time/email	5 ms
Total time/email	6.6 ms
Total time/hour	52.8 seconds

RSA contribution 75.76%

AES contribution 24.24%

Conclusion

The total encryption time is 6.6 ms per email, with RSA key exchange being the main bottleneck. Using ECC, parallel processing, and hardware acceleration can significantly reduce latency while maintaining strong security.